

Hands-On HttpListener in C#

SWEN1 | BIF3

What is HttpListener?

HttpListener is a low-level, Windows-only, built-in HTTP server API in .NET that lets you accept HTTP(S) requests directly in a console app—without ASP.NET. You bind URL prefixes, start listening, and handle `HttpListenerContext` objects (request + response).

In the following slides there is an introduction, core concepts for a small but robust web server using only HttpListener.

Documentation:

- Official Reference: https://learn.microsoft.com/de-de/dotnet/api/system.net.httplistener
- Learn HttpListener: https://learn.microsoft.com/de-de/dotnet/fundamentals/runtime-libraries/system-net-httplistener

Samples:

- https://git.technikum-wien.at/swen/swen1/csharp/httpserverdemo

Introduction

`HttpListener` (`namespace System.Net`) is a managed wrapper over the Windows HTTP Server API (http.sys). It lets a .NET process:

- Reserve one or more URL prefixes (e.g., http://localhost:8080/, https://+:8443/).
- Accept TCP connections and HTTP requests via the OS kernel-mode queue.
- Read `HttpListenerRequest` (method, URL, headers, body) and write `HttpListenerResponse` (status code, headers, body).
- Support HTTP/1.1 features such as keep-alive, chunked transfer encoding, range requests (served manually), and kernel caching (limited).
- Optionally require authentication.

Typical use cases: Lightweight admin endpoints, test servers, simple embedded services, or teaching low-level HTTP.

Pre-Requisites

1. **URL Prefixes** - Optional, default settings already applied
   You must register one or more prefixes, e.g.:
   - http://localhost:8080/ — loopback only
   - http://+:8080/ — all IPs on port 8080
   - https://myhost:8443/api/ — HTTPS with SNI/cert binding
     A prefix must end with a trailing slash.
2. **Permissions** (URL ACLs) - Optional, default settings already applied
   The process must be allowed to listen on the chosen prefixes. For non-admin processes, register an ACL:

```
netsh http add urlacl url=http://+:8080/ user=DOMAIN\user
```

   Remove with:

```
netsh http delete urlacl url=http://+:8080/
```

Lifecycle

Lifecycle:

- Create `HttpListener`, add prefixes, call `Start()`.
- Loop on `GetContextAsync()` to receive `HttpListenerContext`.
- Handle `context.Request` and write to `context.Response`.
- Close/Abort responses; stop the listener on shutdown.

Concurrency:
Each `GetContextAsync()` yields a single request. Dispatch to worker tasks to process multiple requests concurrently.

Graceful shutdown:
Use `CancellationToken` and stop accepting new contexts while finishing in-flight requests.

Requests & Responses

Requests (`HttpListenerRequest`):

- `HttpMethod`, `Url`, `Headers`, `InputStream`, `ContentLength64`, `HasEntityBody`.
- Supports query strings, cookies (`Cookies`), client certificates (`GetClientCertificate()` for HTTPS + mutual auth).

Responses (`HttpListenerResponse`)

- `StatusCode`, `StatusDescription`, `ContentType`, `OutputStream`, `Headers`, `KeepAlive`, `SendChunked`.
- Set `ContentLength64` when known; otherwise enable `SendChunked = true`.

Authentication (Optional):

- `AuthenticationSchemes` (Anonymous, Basic, NTLM, Negotiate).
- `AuthenticationSchemeSelectorDelegate` for per-request selection.

SimpleHttpServer - Demo

```
var listener = new HttpListener();
listener.Prefixes.Add("http://localhost:8080/");
listener.Start();
Console.WriteLine("Listening on http://localhost:8080/  (Press Ctrl+C to exit)");

while (true)
{
    var ctx = await listener.GetContextAsync(); // Accept a request
    _ = Task.Run(async () =>
    {
        var req = ctx.Request;
        var resp = ctx.Response;

        var body = $"Hello from HttpListener! You requested: {req.HttpMethod} {req.Url}\n";
        var bytes = Encoding.UTF8.GetBytes(body);

        resp.StatusCode = 200;
        resp.ContentType = "text/plain; charset=utf-8";
        resp.ContentLength64 = bytes.Length;
        await resp.OutputStream.WriteAsync(bytes, 0, bytes.Length);
        resp.Close();
    });
}

// finally (ATTENTION: try/catch and error-handlig omitted here...)
listener.Stop(); listener.Close();
```

Error handling, timeouts, and robustness

- Wrap each handler in `try/catch`; always try to `Close()` the response.
- Consider `response.Headers["Connection"] = "close"` for fatal errors.
- Use `listener.TimeoutManager` to tune `EntityBody`, `DrainEntityBody`, `RequestQueue`, etc.
- For long-running operations, stream progress with chunked encoding to avoid timeouts.

Small access logger (optional)

```
var sw = System.Diagnostics.Stopwatch.StartNew();
// ... handle request
sw.Stop();
Console.WriteLine($"{ctx.Request.RemoteEndPoint} \" \
    {ctx.Request.HttpMethod} {ctx.Request.Url}\" {ctx.Response.StatusCode} {sw.ElapsedMilliseconds}ms");
```

Enabling HTTPS (self-hosted TLS)

1. Install or create a certificate (in Local Computer → Personal store). Get its thumbprint.
2. Bind the certificate to a port:

```
netsh http add sslcert ipport=0.0.0.0:8443 certhash=THUMBPRINT appid={00000000-0000-0000-0000-000000000000}
```

   Remove with: `netsh http delete sslcert ipport=0.0.0.0:8443`

3. Add an HTTPS prefix:

```
listener.Prefixes.Add("https://+:8443/");
```

4. Optionally require client certs:

```
listener.AuthenticationSchemes = AuthenticationSchemes.Anonymous;
listener.IgnoreWriteExceptions = true; // useful for client aborts
// In request handler:
var cert = ctx.Request.GetClientCertificate();
```

Note: On HTTPS you still need URL ACLs for the prefix and the SSL cert binding above.

Further Reading

- Official Reference: https://learn.microsoft.com/de-de/dotnet/api/system.net.httplistener
- Learn HttpListener: https://learn.microsoft.com/de-de/dotnet/fundamentals/runtime-libraries/system-net-httplistener

Samples:

- https://git.technikum-wien.at/swen/swen1/csharp/httpserverdemo